# 09 中位數與次序統計量 解答

## 09-E01 同時最小最大比較

### 推導

先以首元素初始化 min=max，剩下 10 個元素分成 5 對，每對先比較一次找 pair min/max，共 5 次；每對再各更新全域 min 與 max，最多 2 次，共 10 次，總 15 次。

### 結論與檢核

公式 ceil(3n/2)-2=ceil(16.5)-2=17-2=15。首元素已同時作為兩者的初始界，所以不需與自己比較。

## 09-E02 隨機 select 的區間轉換

### 推導

pivot 整體秩為 5。k=7>5，答案在右側。右側元素整體秩為 6..10，因此要找右側局部第 7-5=2 小。

### 結論與檢核

若 k=3<5，答案在左側並保持局部第 3 小。若 k=5 直接回傳 pivot。區間轉換的重點是扣除已確定較小的元素與 pivot。

## 09-E03 中位數的中位數排除量

### 推導

g=ceil(n/5)個組median中，各至少ceil(g/2)個<=x與>=x。保守扣殘組及pivot組後，各剩至少ceil(g/2)−2個完整組，每組至少提供3筆非嚴格大小證據。因此各至少 3(ceil(g/2)−2)>=3n/10−6。

### 結論與檢核

n=50時g=10，扣殘組與pivot組的保守界為3(5−2)=9。合格完整組是至少貢獻3筆非嚴格大小證據，不是至多。三路版本往嚴格側遞迴，可排除整個另一側加等值區，所以重複鍵亦適用。

## 09-E04 含重複鍵與非零起點的局部秩

### 推導

p=4，所以L=lt−p=1、E=gt−lt+1=3，局部等值秩是2..4。k1往A[4..4]仍找第1小；k4直接回3；k6往A[8..10]改找第6−1−3=2小。

# 09 中位數與次序統計量 解答 （續）

### 結論與檢核

lt/gt是絕對0-index位置，k是局部1-index秩，兩者不同。若先轉成全陣列絕對目標位置 p+k−1，也可始終保留該絕對位置，但不能再扣局部秩。

## 09-E05 Median-of-medians不是精確Median

### 推導

組median為4,8,12,14,15，lower median=12。全體恰為1..25，所以小於12有11筆、等值1筆、大於13筆。第16小在大側，局部秩16−11−1=4。真正median是第13小=13，並非12。

### 結論與檢核

Pivot只是以線性成本取得分割比例保證，沒有保證一次命中全體median。前三組各3筆 <=12、後三組各3筆>=12可直接列出9筆證據；重疊pivot不影響分支各自的排除界。

## 09-E06 平均較大側與期望歸納

### 推導

六個大小為5,4,3,3,4,5，和24，平均4=3n/4−1/2。一般平均<=3n/4，所以條件於pivot並對各子問題套歸納，有F<=cn+3an/4。選a>=4c並涵蓋基例，即F<=an。

### 結論與檢核

不能把E[T(S)]直接換成T(E[S])，非線性函數通常不滿足這個等式。證明是在每種實現S上先套T期望<=aS，再用期望線性性；允許重複鍵時，以identity總序提供同樣大小上界。

# 09 中位數與次序統計量 解答 （續）

## 原創驗算程式附錄: selection_checks.py

```
"""Exact order-statistic oracles, median witnesses, and finite expectation sums."""
from functools import cache
from fractions import Fraction
from itertools import product, combinations_with_replacement
import random
import sys
from pathlib import Path
sys.path.insert(0,str(Path(__file__).resolve().parent))
from selection_partitions import partition3


GROUPS=[[1,2,4,17,25],[3,5,8,18,24],[6,9,12,19,23],[7,10,14,20,22],[11,13,15,16,21]]


def minmax(values):
    n=len(values)
    if not n:
        raise ValueError("nonempty input required")
    if n%2:
        lo=hi=values[0]
        start=1
        comparisons=0
    else:
        lo,hi=(values[0],values[1]) if values[0]<=values[1] else (values[1],values[0])
        start=2
        comparisons=1
    for i in range(start,n,2):
        a,b=values[i:i+2]
        small,large=(a,b) if a<=b else (b,a)
        comparisons+=1
        if small<lo:
            lo=small
        if large>hi:
            hi=large
        comparisons+=2
    return lo,hi,comparisons


def random_select(values,k,rng,prefix=()):
    if not 1<=k<=len(values):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 09 中位數與次序統計量 解答 （續）

```
        raise ValueError("local1-index rank outside input")
    data=list(prefix)+list(values)
    p=len(prefix)
    r=len(data)-1
    while p<r:
        pivot=data[rng.randrange(p,r+1)]
        lt,i,gt=p,p,r
        while i<=gt:
            if data[i]<pivot:
                data[lt],data[i]=data[i],data[lt]
                lt+=1
                i+=1
            elif data[i]>pivot:
                data[i],data[gt]=data[gt],data[i]
                gt-=1
            else:
                i+=1
        left=lt-p
        equal=gt-lt+1
        if k<=left:
            r=lt-1
        elif k<=left+equal:
            return pivot
        else:
            k-=left+equal
            p=gt+1
    return data[p]


def deterministic_select(values,k,trace=None):
    """Readable list implementation: linear extra storage, not an in-place claim."""
    n=len(values)
    if not 1<=k<=n:
        raise ValueError("local1-index rank outside input")
    if n<=5:
        answer=sorted(values)[k-1]
        if trace is not None:
            trace.append((n,k,"base",answer))
        return answer
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 09 中位數與次序統計量 解答 （續）

```
    groups=[sorted(values[i:i+5]) for i in range(0,n,5)]
    medians=[group[(len(group)-1)//2] for group in groups]
    pivot=deterministic_select(medians,(len(medians)+1)//2)
    left=[v for v in values if v<pivot]
    equal=[v for v in values if v==pivot]
    right=[v for v in values if v>pivot]
    assert min(n-len(left),n-len(right))>=Fraction(3*n,10)-6
    if trace is not None:
        trace.append((n,k,medians,pivot,(len(left),len(equal),len(right))))
    if k<=len(left):
        return deterministic_select(left,k,trace)
    if k<=len(left)+len(equal):
        return pivot
    return deterministic_select(right,k-len(left)-len(equal),trace)


@cache
def expected_scan_work(values,k):
    """Enumerate every equally probable pivot identity, retaining duplicate
multiplicities."""
    n=len(values)
    if n==1:
        return Fraction(1)
    future=Fraction(0)
    for pivot in values:
        left=tuple(v for v in values if v<pivot)
        equal=sum(v==pivot for v in values)
        right=tuple(v for v in values if v>pivot)
        if k<=len(left):
            future+=expected_scan_work(left,k)
        elif k>len(left)+equal:
            future+=expected_scan_work(right,k-len(left)-equal)
    return n+future/n


def main():
    arrays=0
    ranks=0
    rng=random.Random(20261024)
    for n in range(1,8):
        for values in product(range(3),repeat=n):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 09 中位數與次序統計量 解答 （續）

```
            lo,hi,count=minmax(values)
            assert (lo,hi)==(min(values),max(values))
            assert count==(3*n+1)//2-2
            reference=sorted(values)
            for k in range(1,n+1):
                assert deterministic_select(values,k)==reference[k-1]
                assert random_select(values,k,rng,prefix=(999,-999))==reference[k-1]
                ranks+=1
            arrays+=1
    for n in [8,13,25,50,140,141,503,1001]:
        for _ in range(10):
            values=[rng.randrange(-n,n) for _ in range(n)]
            for k in {1,n,(n+1)//2,rng.randrange(1,n+1)}:
                assert deterministic_select(values,k)==sorted(values)[k-1]
    trace=[]
    answer=deterministic_select([v for group in GROUPS for v in group],16,trace)
    assert answer==16
    assert trace==[(25,16,[4,8,12,14,15],12,(11,1,13)),(13,4,[19,20,16],19,(6,1,6)),(6,4,
[15,16],15,(2,1,3)),(3,1,'base',16)]
    for n in range(1,501):
        assert Fraction(sum(max(r-1,n-r) for r in range(1,n+1)),n)<=Fraction(3*n,4)
    exact_states=0
    for n in range(1,10):
        for values in combinations_with_replacement(range(3),n):
            for k in range(1,n+1):
                assert expected_scan_work(values,k)<=4*n
                exact_states+=1
    for n in range(2,31):
        values=tuple(range(n))
        for k in range(1,n+1):
            assert expected_scan_work(values,k)<=4*n
            exact_states+=1
    assert partition3([8,3,5,3,9,1,3],3)==([1,3,3,3,9,5,8],1,3)
    print(f"PASS selection_arrays={arrays} all_ranks={ranks} exact_expected_states=
{exact_states} nonzero_start_rank=True n25_trace={trace} paired_minmax_count_formula=True")


if __name__=="__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 09 中位數與次序統計量 解答 （續）

## 原創驗算程式附錄: selection_partitions.py

```
"""Check three-way selection semantics and the all-equal Lomuto counterexample."""


def partition3(a, pivot):
    a = list(a)
    lt, i, gt = 0, 0, len(a) - 1
    while i <= gt:
        if a[i] < pivot:
            a[lt], a[i] = a[i], a[lt]
            lt += 1
            i += 1
        elif a[i] > pivot:
            a[i], a[gt] = a[gt], a[i]
            gt -= 1
        else:
            i += 1
    return a, lt, gt


def lomuto_comparisons(n):
    return n * (n - 1) // 2


def run():
    values = [3, 3, 1, 3, 2, 3]
    ordered, lt, gt = partition3(values, 3)
    assert ordered == [1, 2, 3, 3, 3, 3]
    assert (lt, gt) == (2, 5)
    target = 3
    assert lt <= target <= gt and ordered[target] == 3
    all_equal, lt2, gt2 = partition3([5] * 5, 5)
    assert all_equal == [5] * 5 and (lt2, gt2) == (0, 4)
    assert lomuto_comparisons(5) == 10
    assert lomuto_comparisons(10) == 45
    print("PASS partitions three_way=(2,5) all_equal=(0,4) lomuto_n5=10")
    return ordered, (lt, gt)


if __name__ == "__main__":
    run()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

**參考資料與署名**

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study